1. Design and Implementation of Pass 1 of two Pass Macro Processor:

```
n = int(input("Enter number of source program lines: "))
program = [input() for _ in range(n)]

mnt = []   # Macro Name Table: (name, mdt_index)
mdt = []   # Macro Definition Table

i = 0
while i < n:
    word = program[i].split()[0]

    if word == "MACRO":
        i += 1
        name = program[i].split()[0]

        mnt.append((name, len(mdt)))
        mdt.append(program[i])

        i += 1
        while not program[i].startswith("MEND"):
            mdt.append(program[i])
            i += 1

        mdt.append("MEND")
    i += 1

# Output
print("\nMNT TABLE")
print("Macro Name\tMDT Index")
for name, idx in mnt:
    print(f"{name}\t\t{idx}")

print("\nMDT TABLE")
for i, line in enumerate(mdt):
    print(f"{i}\t{line}")
```

OUTPUT:

```
Enter number of source program lines: 4
MACRO
INCR
A 1
MEND

MNT TABLE
Macro Name      MDT Index
INCR            0

MDT TABLE
0       INCR
1       A 1
2       MEND
```

2. Design and Implementation of Pass 2 of two Pass Macro Processor:

```
# Input MNT
mntc = int(input("Enter number of MNT entries: "))
mnt = {}
for _ in range(mntc):
    name, idx = input().split()
    mnt[name] = int(idx)

# Input MDT
mdtc = int(input("Enter number of MDT entries: "))
mdt = [input() for _ in range(mdtc)]

# Input Source Program
n = int(input("Enter number of source lines: "))
source = [input() for _ in range(n)]

print("\nEXPANDED SOURCE PROGRAM")

for line in source:
    parts = line.split()

    if parts and parts[0] in mnt:
        macro = parts[0]
        arg = parts[1] if len(parts) > 1 else ""

        k = mnt[macro] + 1

        while not mdt[k].startswith("MEND"):
            temp = mdt[k]

            if "&ARG" in temp:
                temp = temp.replace("&ARG", arg)

            print(temp)
            k += 1
    else:
        print(line)
```

OUTPUT:

```
Enter number of MNT entries: 1
INCR 0
Enter number of MDT entries: 3
INCR
ADD &ARG
MEND
Enter number of source lines: 2
INCR 5
START

EXPANDED SOURCE PROGRAM
ADD 5
START
```

3. Study and Implementation of Lexical Analyzer:

```
exp = input("Enter expression: ")
stack = []

for ch in exp:
    if ch.isalpha():
        print(f"Identifier: {ch}")
        stack.append(ch)
    elif ch.isdigit():
        print(f"Number: {ch}")
    elif ch in "+-*/=":
        print(f"Operator: {ch}")
    else:
        print(f"Symbol: {ch}")

print("\nSymbol Table")
print("ID\tVariable\tType")
for i, var in enumerate(stack, 1):
    print(f"{i}\t{var}\t\tint")
```

OUTPUT:

```
Enter expression: a = b + 5
Identifier: a
Symbol:
Operator: =
Symbol:
Identifier: b
Symbol:
Operator: +
Symbol:
Number: 5

Symbol Table
ID      Variable        Type
1       a               int
2       b               int
```

4. Implementation of Recursive Decent Parser. (Top-down Parser):

```
input_str = input("Enter input string: ")
pos = 0
error = False

class Node:
    def __init__(self, name):
        self.name = name
        self.children = []

def E():
    node = Node("E")
    node.children.append(T())
    node.children.append(Ep())
    return node

def Ep():
    global pos
    node = Node("E'")

    if pos < len(input_str) and input_str[pos] == '+':
        pos += 1
        node.children.append(Node("+"))
        node.children.append(T())
        node.children.append(Ep())
    else:
        node.children.append(Node("ε"))

    return node

def T():
    node = Node("T")
    node.children.append(F())
    return node

def F():
    global pos, error
    if pos < len(input_str) and input_str[pos] == 'i':
        pos += 1
        return Node("i")
    else:
        error = True
        return Node("error")

def print_tree(node, depth=0):
    if not node:
        return
    print("   " * depth + "|-- " + node.name)
    for child in node.children:
```

```
        print_tree(child, depth + 1)

# Main
root = E()

if not error and pos == len(input_str):
    print("\nString ACCEPTED\n")
    print("Parse Tree:")
    print_tree(root)
else:
    print("\nString REJECTED")
```

OUTPUT:

```
Enter input string: i+i+i

String ACCEPTED

Parse Tree:
|-- E
   |-- T
      |-- i
   |-- E'
      |-- +
      |-- T
         |-- i
      |-- E'
         |-- +
         |-- T
            |-- i
         |-- E'
            |-- ε
```

5. Implementation of Intermediate code generation phase of compiler:

```
def precedence(op):
    return 1 if op in "+-" else 2 if op in "*/" else 0

def infix_to_postfix(exp):
    stack, postfix = [], []

    for ch in exp:
        if ch.isalnum():
            postfix.append(ch)
        elif ch == '(':
            stack.append(ch)
        elif ch == ')':
```

```
            while stack and stack[-1] != '(':
                postfix.append(stack.pop())
            stack.pop()
        else:
            while stack and precedence(stack[-1]) >= precedence(ch):
                postfix.append(stack.pop())
            stack.append(ch)

    while stack:
        postfix.append(stack.pop())

    return ''.join(postfix)

def generate_icg(postfix):
    stack = []
    temp_count = 1
    quads = []

    print("\nThree Address Code:")

    for ch in postfix:
        if ch.isalnum():
            stack.append(ch)
        else:
            op2 = stack.pop()
            op1 = stack.pop()
            temp = f"T{temp_count}"
            temp_count += 1

            print(f"{temp} = {op1} {ch} {op2}")
            quads.append((ch, op1, op2, temp))
            stack.append(temp)

    print("\nQuadruple Representation:")
    print("Op\tArg1\tArg2\tResult")
    for q in quads:
        print(f"{q[0]}\t{q[1]}\t{q[2]}\t{q[3]}")

    print("\nTriple Representation:")
    print("Index\tOp\tArg1\tArg2")
    for i, q in enumerate(quads):
        print(f"{i}\t{q[0]}\t{q[1]}\t{q[2]}")

    print("\nIndirect Triple Representation:")
    for i in range(len(quads)):
        print(f"P{i} -> {i}")

# Main
infix = input("Enter Infix Expression: ")
postfix = infix_to_postfix(infix)
print("\nPostfix Expression:", postfix)
generate_icg(postfix)
```

OUTPUT:

```
Enter Infix Expression: abc*+

Postfix Expression: abc*+

Three Address Code:
T1 = b * c
T2 = a + T1

Quadruple Representation:
Op      Arg1    Arg2    Result
*       b       c       T1
+       a       T1      T2

Triple Representation:
Index   Op      Arg1    Arg2
0       *       b       c
1       +       a       T1

Indirect Triple Representation:
P0 -> 0
P1 -> 1
```

6. Implementation of code optimization phase of compiler(any 4)

1.) Common Subexpression Elimination

```
n = int(input("Enter number of expressions: "))
expr = [input() for _ in range(n)]

optimized = []

for e in expr:
    if e in optimized:
        print(f"Eliminating redundant expression: {e}")
    else:
        optimized.append(e)

print("\nOptimized Code:")
for e in optimized:
    print(e)
```

OUTPUT:

```
Enter number of expressions: 4
t1=a+b
t2=c+d
t1=a+b
t3=x+y
Eliminating redundant expression: t1=a+b

Optimized Code:
t1=a+b
t2=c+d
t3=x+y
```

2.) Constant Folding

```
expr = input("Enter expression (e.g., t1=5+3): ")
lhs, rhs = expr.split('=')
# extract numbers and operator
import re
a, op, b = re.findall(r'(\d+)([+\-*/])(\d+)', rhs)[0]
a, b = int(a), int(b)
print(f"\nOriginal Expression: {lhs}={a}{op}{b}")
if op == '+':
    res = a + b
elif op == '-':
    res = a - b
elif op == '*':
    res = a * b
elif op == '/':
    res = a / b if b != 0 else "Error (div by 0)"
else:
    res = "Invalid"
print(f"Optimized Expression: {lhs}={res}")
```

OUTPUT:

```
Enter expression (e.g., t1=5+3): t1=8+4

Original Expression: t1=8+4
Optimized Expression: t1=12
```

3.) Strength Reduction

```
# Strength Reduction (Python)
expr = input("Enter expression (e.g., t1=x*2): ")
lhs, rhs = expr.split('=')
var, num = rhs.split('*')
num = int(num)
print(f"\nOriginal Expression: {lhs}={var}*{num}")
if num in [2, 4, 8]:
    shift = {2:1, 4:2, 8:3}[num]
    print(f"Optimized Expression: {lhs}={var}<<{shift}")
else:
    print("No strength reduction possible")
```

OUTPUT:

```
Enter expression (e.g., t1=x*2): t1=x*4

Original Expression: t1=x*4
Optimized Expression: t1=x<<2
```

4.) Dead Code Elimination

```
n = int(input("Enter number of statements: "))
stmt = [input() for _ in range(n)]

lhs, rhs = [], []
for s in stmt:
    l, r = s.split('=')
    lhs.append(l)
    rhs.append(r)

live = set()
dead = [0] * n

# last statement is always useful
live.add(lhs[-1])
for ch in rhs[-1]:
    if ch.isalpha():
        live.add(ch)

# backward scan
for i in range(n - 2, -1, -1):
    if lhs[i] not in live:
        dead[i] = 1
    else:
        live.discard(lhs[i])
        for ch in rhs[i]:
            if ch.isalpha():
                live.add(ch)

print("\nOriginal Code:")
for s in stmt:
    print(s)

print("\nOptimized Code:")
for i in range(n):
    if not dead[i]:
        print(stmt[i])
```

OUTPUT:

```
Enter number of statements: 4
a=8
b=4
a=16
c=a+b

Original Code:
a=8
b=4
a=16
c=a+b

Optimized Code:
b=4
a=16
c=a+b
```